浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：May 8, 2026
地点：玉泉高分子楼-203

课程名称：人工智能实验　指导老师：魏准　实验类型：设计型实验
实验名称：神经网络与深度学习训练实验　成　绩：　签　名：

装 订 线

一、 实验 5-2：BP 训练多层神经网络

1. 实验目的

本实验根据 Topic 5 Part 2 中的要求，使用 SGD 训练方法、Sigmoid 激活函数及 BP 规则训练含隐藏层的神经网络，并与实验 5-1 中的单层 SGD 网络在同一 XOR 数据上的结果进行对比，分析多层网络相对单层网络的优势。

2. 实验原理

实验 5-2 使用 XOR 数据集。输入第三维为偏置项 $x_3 = 1$，标签为异或输出，如表 1 所示。本实现将偏置吸收到输入特征 $x_3 = 1$ 中，因此 W_1 的第三列扮演隐藏层偏置的角色；为简化，隐藏层输出 $y^{(1)}$ 后未再追加常数 1，因而 W_2 不含显式输出层偏置。对本实验的 XOR 任务，隐藏层非线性已经足以表达所需边界，最终 MSE 收敛到 10^{-5} 量级也说明输出层偏置缺失不影响该任务可解性。

x_1	x_2	x_3（偏置）	d
0	0	1	0
0	1	1	1
1	0	1	1
1	1	1	0

Table 1: 实验 5-2 XOR 训练数据

多层网络结构为 $3 \to 4 \to 1$。输入层含两个特征与一个偏置项，隐藏层含 4 个 Sigmoid 神经元，输出层含 1 个 Sigmoid 神经元。前向传播为：

$$v^{(1)} = W_1 x, \qquad y^{(1)} = \sigma(v^{(1)})$$

$$v^{(2)} = W_2 y^{(1)}, \qquad y = \sigma(v^{(2)})$$

其中：

$$\sigma(v) = \frac{1}{1+e^{-v}}, \qquad \sigma'(v) = \sigma(v)(1-\sigma(v))$$

输出层与隐藏层 Delta 分别为：

$$e = d - y, \qquad \delta^{(2)} = y \odot (1-y) \odot e$$

$$e^{(1)} = W_2^T \delta^{(2)}, \qquad \delta^{(1)} = y^{(1)} \odot (1 - y^{(1)}) \odot e^{(1)}$$

其中 $\odot$ 表示逐元素相乘；本实验输出层只有 1 个神经元，因此输出层公式退化为标量乘法。

权重采用 SGD 逐样本更新：

$$W_2 \leftarrow W_2 + \alpha \delta^{(2)} (y^{(1)})^T, \qquad W_1 \leftarrow W_1 + \alpha \delta^{(1)} x^T$$

3. 代码实现

Listing 1 给出 3-4-1 Sigmoid 网络的一轮 BP 更新过程，包括前向计算、输出层与隐藏层 Delta 计算以及权重更新；Listing 2 给出 XOR 数据上的单层 SGD 与多层 BP 训练、预测、MSE 和准确率计算函数。

Listing 1: BP.py：3-4-1 Sigmoid 网络的 BP 训练函数

```python
import numpy as np

from Sigmoid import Sigmoid


def Backprop(W1, W2, X, D, alpha=1.0):
    """One SGD epoch for a 3-4-1 sigmoid network."""
    n_samples = X.shape[0]
    order = np.random.permutation(n_samples)

    for k in order:
        x = X[k]
        d = D[k]

        v1 = W1 @ x
        y1 = Sigmoid(v1)
        v2 = (W2 @ y1)[0]
        y = Sigmoid(v2)

        e = d - y
        delta2 = y * (1.0 - y) * e

        delta1 = y1 * (1.0 - y1) * (W2.T @ np.atleast_1d(delta2)).ravel()

        dW1 = alpha * np.outer(delta1, x)
        W1 = W1 + dW1

        dW2 = alpha * delta2 * y1.reshape(1, -1)
        W2 = W2 + dW2

    return W1, W2
```

Listing 2: exp5_2.py：多层网络与单层网络训练及评价函数

```python
def single_sgd_epoch(W, X, D, alpha=1.0):
    order = np.random.permutation(X.shape[0])
    for k in order:
        x = X[k]
        d = D[k]
        y = Sigmoid((W @ x)[0])
        e = d - y
        delta = y * (1.0 - y) * e
        W = W + alpha * delta * x.reshape(1, -1)
    return W


def predict_bp(W1, W2, X):
    hidden = Sigmoid(X @ W1.T)
```

```
    return Sigmoid(hidden @ W2.T).reshape(-1)
def predict_single(W, X):    return Sigmoid(X @ W.T).reshape(-1)


def mse(y, d):
    return float(np.mean((d - y) ** 2))


def accuracy(y, d):
    return float(np.mean((y >= 0.5).astype(int) == d.astype(int)))


def train_bp(seed=7, epochs=50000, alpha=1.0):
    np.random.seed(seed)
    W1 = np.random.uniform(-1.0, 1.0, (4, 3))
    W2 = np.random.uniform(-1.0, 1.0, (1, 4))
    history = []

    for _ in range(epochs):
        W1, W2 = Backprop(W1, W2, X, D, alpha=alpha)
        history.append(mse(predict_bp(W1, W2, X), D))

    return W1, W2, np.array(history)


def train_single(seed=7, epochs=50000, alpha=1.0):
    np.random.seed(seed)
    W = np.random.uniform(-1.0, 1.0, (1, 3))
    history = []

    for _ in range(epochs):
        W = single_sgd_epoch(W, X, D, alpha=alpha)
        history.append(mse(predict_single(W, X), D))

    return W, np.array(history)
```

4. 实验结果与分析

实验参数为：随机种子 7，学习率 $\alpha = 1.0$，多层 BP 网络与单层 SGD 网络均训练 50000 epochs，初始权重均从 $[-1, 1]$ 均匀分布采样。由于 XOR 网络规模很小且 Sigmoid 导数有界，$\alpha = 1.0$ 在本任务中仍能稳定更新；较长的 50000 epochs 用于保证两种网络在同一训练预算下公平比较。训练后结果如表 2 所示。

方法	最终 MSE	准确率	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$
多层 BP 网络	0.00001305	100%	0.002771	0.996705	0.995867	0.004073
单层 SGD 网络	0.25007145	75%	0.514734	0.504774	0.505183	0.495220

Table 2: 实验 5-2 多层网络与单层网络最终结果

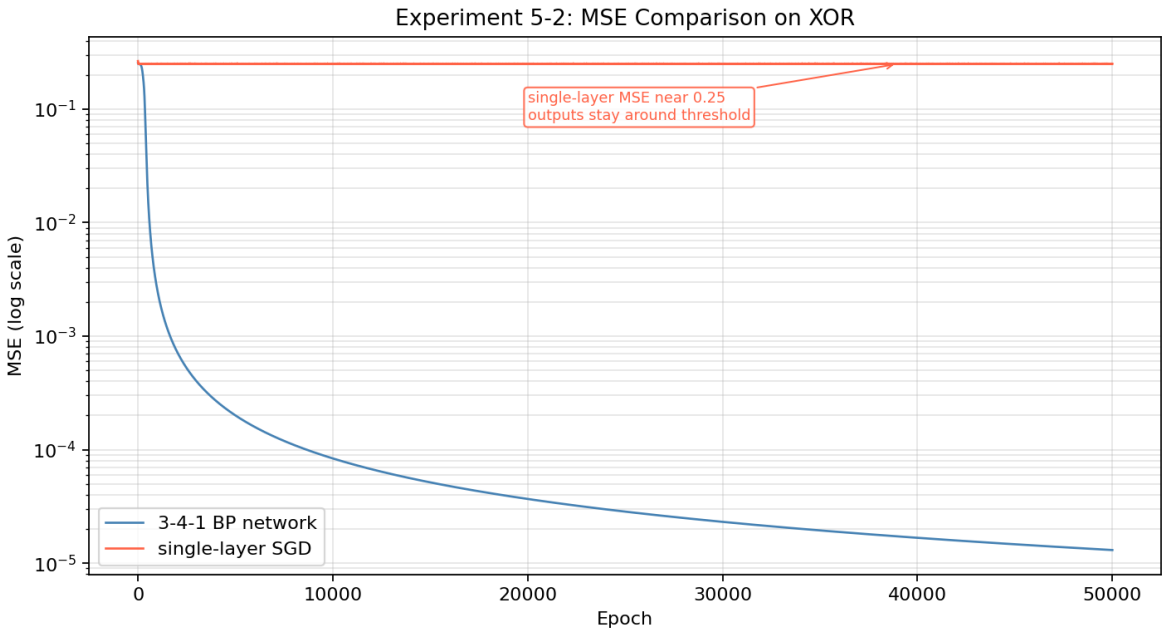


Figure 1: 实验 5-2：多层 BP 网络与单层 SGD 网络的 MSE 曲线

装 订 线

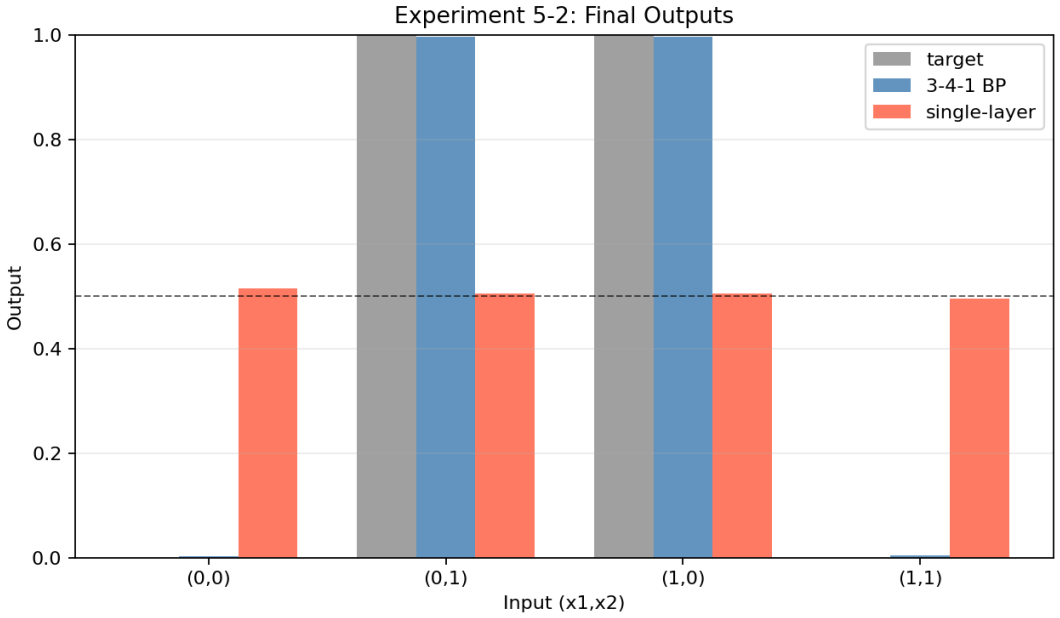


Figure 2: 实验 5-2：四组样本最终输出对比

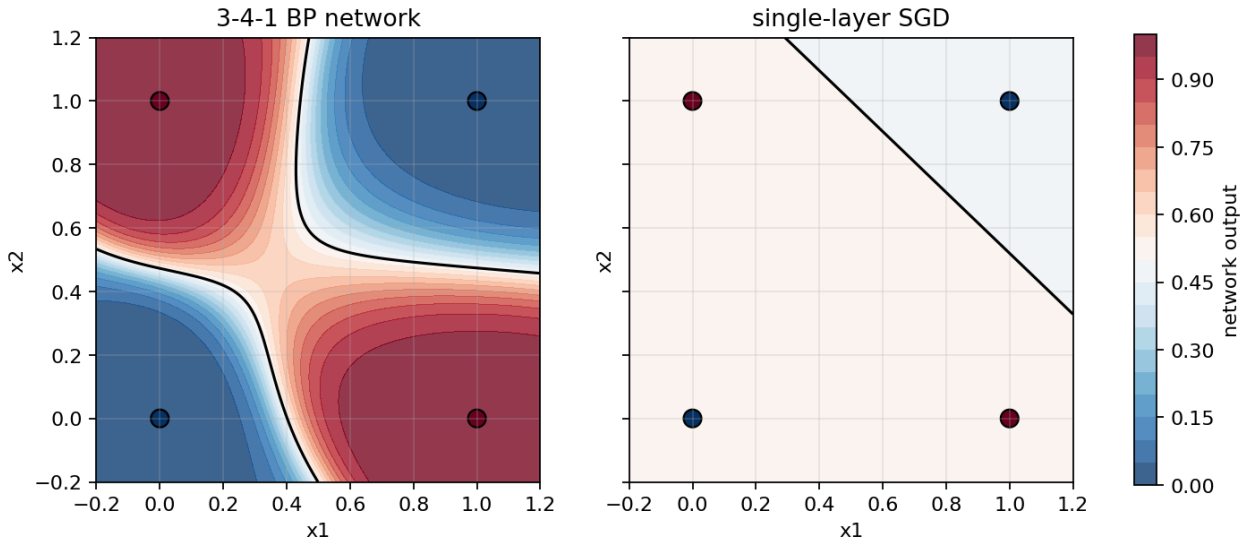


Figure 3: 实验 5-2：多层网络与单层网络决策边界对比

从图 1–3 可以看出，多层 BP 网络能够将 XOR 的四组样本正确分开，而单层网络输出基本停留在 0.5 附近。当四组样本输出都退化到 0.5 时，无论标签为 0 还是 1，平方误差均为 $(0.5)^2 = 0.25$，这解释了单层网络 MSE 曲线长期贴近 0.25 的现象。单层 SGD 网络虽按 0.5 阈值计算得到 75% 的名义准确率，但四个输出值均紧贴 0.5：$(0, 0)$ 的标签为 0，输出 0.514734 越过阈值后被错判为 1，是唯一的错误样本；$(1, 1)$ 的输出 0.495220 恰好低于阈值被判为 0，与标签碰巧一致；其余两个样本标签均为 1，也因略高于阈值而被判对。因此该准确率主要来自阈值碰巧划分，网络实际上没有形成有效区分能力。这正是 XOR 线性不可分的体现；加入隐藏层后，多个非线性神经元可以组合出非线性边界，从而成功学习 XOR 映射。

装 订 线

二、 实验 5-3：Momentum 训练多层神经网络

1. 实验目的

本实验在实验 5-2 的多层神经网络基础上，将 BP 的权重更新规则改为 momentum 更新规则，比较不同 β 值 momentum 训练方法及不用 momentum 训练方法时，误差（真实结果与输出的 MSE）随 epoch 的变化趋势，并可视化结果。

2. 实验原理

普通 BP 的权重增量为：

$$\Delta W_t = \alpha \delta y^T, \qquad W_t = W_{t-1} + \Delta W_t$$

Momentum 方法在当前梯度方向之外保留历史更新方向。设 m_t 为第 t 步动量项：

$$m_t = \beta m_{t-1} + \Delta W_t$$

$$W_t = W_{t-1} + m_t$$

其中 β 为 momentum 系数。若连续若干步的梯度方向近似一致，设 $\Delta W_t = \alpha g_t$，则动量项相当于对历史梯度作几何加权：

$$m_t \approx \left(1 + \beta + \beta^2 + \cdots\right) \alpha g_t = \frac{\alpha}{1-\beta} g_t$$

因此可用近似等效学习率

$$\alpha_{\text{eff}} \approx \frac{\alpha}{1-\beta}$$

理解 momentum 的放大作用。适当的 β 可以在稳定下降方向上加速训练；但当 β 过大时，历史方向占比过高，等效步长也会显著增大，可能导致参数越过较优区域并产生振荡。

3. 代码实现

Listing 3 在普通 BP 更新的基础上加入 momentum 项，保存并累积历史权重增量；Listing 4 给出不同 β 设置下的初始化、预测、评价指标和训练函数，用于对比不用 momentum 与使用 momentum 的训练效果。

装 订 线

Listing 3: BackPropMmt.py：带 momentum 的 BP 更新函数

```python
import numpy as np

from Sigmoid import Sigmoid


def BackPropMmt(W1, W2, X, D, mmt1=None, mmt2=None, alpha=1.0, beta=0.9):
    """One SGD epoch for a 3-4-1 sigmoid network with momentum."""
    if mmt1 is None:
        mmt1 = np.zeros_like(W1)
    if mmt2 is None:
        mmt2 = np.zeros_like(W2)

    n_samples = X.shape[0]
    order = np.random.permutation(n_samples)

    for k in order:
        x = X[k]
        d = D[k]

        v1 = W1 @ x
        y1 = Sigmoid(v1)
        v2 = (W2 @ y1)[0]
        y = Sigmoid(v2)

        e = d - y
        delta2 = y * (1.0 - y) * e
        hidden_error = (W2.T @ np.atleast_1d(delta2)).ravel()
        delta1 = y1 * (1.0 - y1) * hidden_error

        dW2 = alpha * delta2 * y1.reshape(1, -1)
        dW1 = alpha * np.outer(delta1, x)

        mmt2 = beta * mmt2 + dW2
        mmt1 = beta * mmt1 + dW1
        W2 = W2 + mmt2
        W1 = W1 + mmt1

    return W1, W2, mmt1, mmt2
```

装 订 线

Listing 4: exp5_3.py：不同 beta 值的训练及评价函数

```python
def predict(W1, W2, X):
    hidden = Sigmoid(X @ W1.T)
    return Sigmoid(hidden @ W2.T).reshape(-1)


def mse(y, d):
    return float(np.mean((d - y) ** 2))


def accuracy(y, d):
    return float(np.mean((y >= 0.5).astype(int) == d.astype(int)))


def initial_weights(seed=7):
    np.random.seed(seed)
    W1 = np.random.uniform(-1.0, 1.0, (4, 3))
    W2 = np.random.uniform(-1.0, 1.0, (1, 4))
    return W1, W2


def train_plain(seed=7, epochs=12000, alpha=1.0):
    W1, W2 = initial_weights(seed)
    history = []
    for _ in range(epochs):
        W1, W2 = Backprop(W1, W2, X, D, alpha=alpha)
        history.append(mse(predict(W1, W2, X), D))
    return W1, W2, np.array(history)


def train_momentum(seed=7, epochs=12000, alpha=1.0, beta=0.9):
    W1, W2 = initial_weights(seed)
    mmt1 = np.zeros_like(W1)
    mmt2 = np.zeros_like(W2)
    history = []
    for _ in range(epochs):
        W1, W2, mmt1, mmt2 = BackPropMmt(W1, W2, X, D, mmt1, mmt2, alpha=alpha, beta=beta)
        history.append(mse(predict(W1, W2, X), D))
    return W1, W2, np.array(history)
```

4. 实验结果与分析

实验参数为：随机种子 7，学习率 $\alpha = 1.0$，训练 12000 epochs，初始权重范围为 $[-1, 1]$。结果如表 3 所示。

方法	MSE	准确率	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$
不用 momentum	0.00006713	100%	0.00653020	0.99270968	0.99055666	0.00914063
$\beta = 0.5$	0.00002859	100%	0.00410933	0.99517306	0.99416786	0.00633815
$\beta = 0.9$	0.00000587	100%	0.00180434	0.99784581	0.99779816	0.00327517
$\beta = 0.99$	0.18749994	75%	0.00027471	0.50000081	0.49999950	0.49999999

Table 3: 实验 5-3 不同 momentum 设置下的最终结果

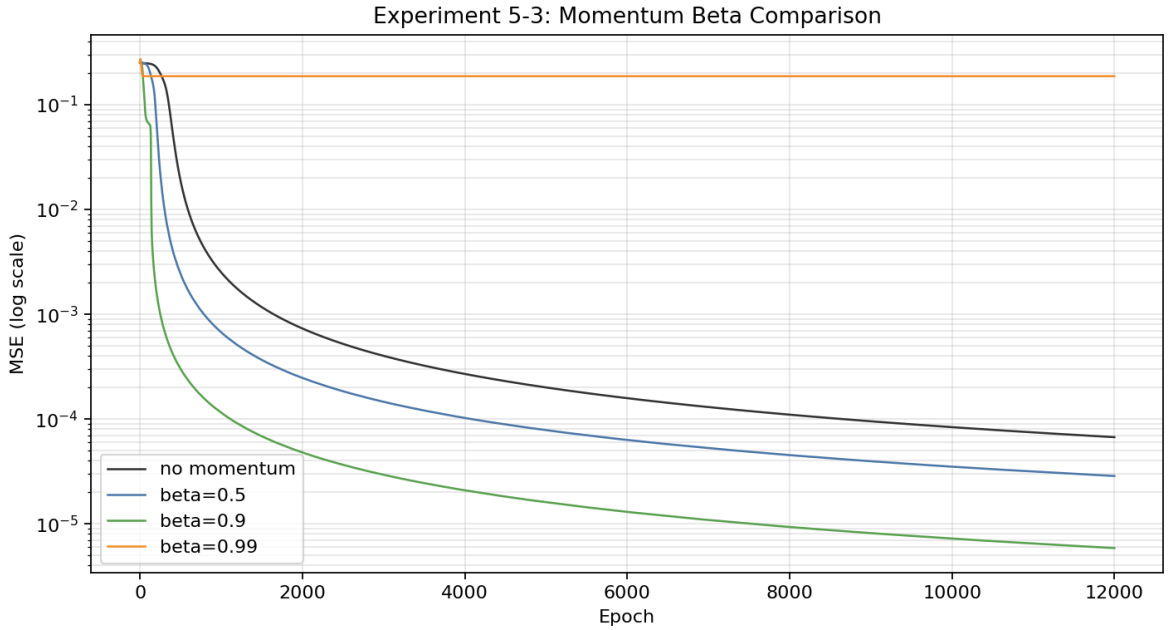


Figure 4: 实验 5-3：不同 β 值与不用 momentum 的 MSE 曲线

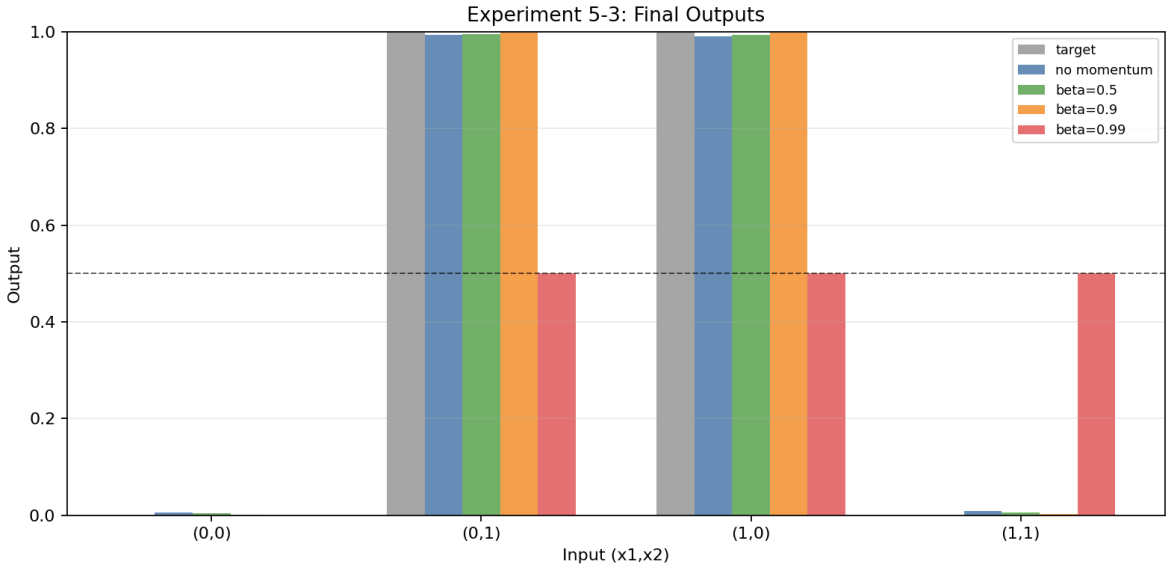


Figure 5: 实验 5-3：不同训练方法下四组 XOR 样本最终输出

由图 4 和图 5 可知，适当的 momentum 能明显改善 BP 训练效果：$\beta = 0.5$ 和 $\beta = 0.9$ 的最终 MSE 均低于不用 momentum 的情形，其中 $\beta = 0.9$ 收敛效果最好。相比之下，$\beta = 0.99$ 的表现显著变差。结合等效学习率近似 $\alpha_{\text{eff}} \approx \alpha/(1-\beta)$ 可知，在本实验 $\alpha = 1.0$ 时，$\beta = 0.99$ 对应的等效步长约为 100，历史更新方向被过度累积，参数容易跨过较优区域并产生振荡，当前梯度难以及时修正方向，因此无法稳定学到 XOR 所需的非线性边界。

从表 3 的最终输出也能看出这种不稳定并不是简单地全部退化到 0.5：$\beta = 0.99$ 已经将 $(0,0)$ 的输出压到接近 0，但其余三组样本仍停留在 0.5 附近。这说明网络较容易拟合由偏置项主导的固定输入模式 $(0,0,1)$，却没有稳定学到 x_1 与 x_2 之间的非线性组合。表中输出保留 8 位小数，是为了避免 $(1,1)$ 的实际输出略小于 0.5 却在

6 位显示下四舍五入为 0.500000，从而造成阈值判别上的误解。进一步将 $\beta = 0.99$ 固定、学习率降为 $\alpha = 0.01$ 后，$\alpha_{\text{eff}} \approx 1$，在相同随机种子和 12000 epochs 下最终 MSE 降至 1.006×10^{-4}，准确率恢复到 100%，输出为 $0.006271, 0.993567, 0.986803, 0.012154$。这进一步说明 $\beta = 0.99$ 本身并非必然失败，关键问题在于过大的 β 与过大的 α 共同放大了实际更新步长。

装 订 线

装 订 线

三、 实验 6-1：训练一个深度学习网络

1. 实验目的

本实验根据 Topic 6 Part 1 的要求，通过 SGD 训练方法、ReLU 激活函数、最后一层 Softmax 以及 BP 规则，对一个深度神经网络进行训练，并输出训练后的分类结果。课件为简单起见指定 Softmax 函数的导数直接取 1；其与常用 Softmax + 交叉熵情形的区别在实验原理中说明。

2. 实验原理

实验 6-1 的网络结构为：

$$25 \to 20 \to 20 \to 20 \to 5$$

对应权重矩阵维度为：

$$W_1 \in \mathbb{R}^{20\times 25}, \quad W_2 \in \mathbb{R}^{20\times 20}, \quad W_3 \in \mathbb{R}^{20\times 20}, \quad W_4 \in \mathbb{R}^{5\times 20}$$

输入为 5 个 5×5 数字图案，展开后为 25 维向量；输出为 5 类 one-hot 标签。数据由 `make_dataset()` 按课件图案逐项定义，例如第 1 个样本矩阵为

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

隐藏层采用 ReLU：

$$\mathrm{ReLU}(x) = \max(0, x), \qquad \mathrm{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \le 0 \end{cases}$$

输出层采用 Softmax：

$$\mathrm{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

反向传播中，课件要求为简单起见令 Softmax 导数为 1，因此输出层 Delta 直接取：

$$\delta = d - y$$

需要注意，若采用 Softmax + 交叉熵损失，输出层相对于 logits 的梯度可精确化为 $y-d$；按本文 $W \leftarrow W+\alpha\delta y^T$ 与 $e = d - y$ 的符号约定，等价于取 $\delta = e$。本实验用 MSE 作为误差曲线与结果指标，因此这里的 $\delta = e$ 是课件指定的近似处理，而不是完整的 Softmax + MSE 导数。

隐藏层依次反传：

$$\delta_3 = \mathrm{ReLU}'(v_3) \odot (W_4^T\delta), \quad \delta_2 = \mathrm{ReLU}'(v_2) \odot (W_3^T\delta_3), \quad \delta_1 = \mathrm{ReLU}'(v_1) \odot (W_2^T\delta_2)$$

权重按 SGD 逐样本更新：

$$W_4 \leftarrow W_4 + \alpha\delta y_3^T, \quad W_3 \leftarrow W_3 + \alpha\delta_3 y_2^T, \quad W_2 \leftarrow W_2 + \alpha\delta_2 y_1^T, \quad W_1 \leftarrow W_1 + \alpha\delta_1 x^T$$

3. 代码实现

Listing 5 定义 ReLU 激活函数及其导数，Listing 6 定义输出层 Softmax 函数；Listing 7 给出深度 ReLU 网络的一轮 SGD-BP 更新过程。Listing 8 给出验证样本与 one-hot 标签的构造，Listing 9 给出 ReLU 与 Sigmoid 两种隐藏层设置下的预测、MSE/accuracy 评价和训练循环，Listing 10 给出最终训练验证与结果输出入口。

Sigmoid 对比网络与 Listing 7 的反向传播结构相同，仅将隐藏层激活函数及其导数由 ReLU 替换为 Sigmoid，因此重点展示其训练调用与 MSE 曲线对比。

Listing 5: ReLU.py：ReLU 及其导数

```python
def ReLU(x):
    return np.maximum(0.0, x)


def ReLU_derivative(x):
    return (x > 0.0).astype(float)
```

Listing 6: Softmax.py：Softmax 函数

```python
import numpy as np


def Softmax(x):
    x = np.subtract(x, np.max(x))
    ex = np.exp(x)
    return ex / np.sum(ex)
```

Listing 7: DeepReLU.py：深度 ReLU 网络训练函数

```python
def DeepReLU(W1, W2, W3, W4, X, D, alpha=0.01):
    """One SGD epoch for a 25-20-20-20-5 deep ReLU network."""
    n_samples = X.shape[2]
    order = np.random.permutation(n_samples)
    for k in order:
        x = np.reshape(X[:, :, k], (25, 1))
        v1 = W1 @ x
        y1 = ReLU(v1)
        v2 = W2 @ y1
        y2 = ReLU(v2)
        v3 = W3 @ y2
        y3 = ReLU(v3)
        v = W4 @ y3
        y = Softmax(v)
        d = D[k, :].reshape(5, 1)
        e = d - y
        delta = e
        e3 = W4.T @ delta
        delta3 = ReLU_derivative(v3) * e3
        e2 = W3.T @ delta3
        delta2 = ReLU_derivative(v2) * e2
        e1 = W2.T @ delta2
        delta1 = ReLU_derivative(v1) * e1
        dW4 = alpha * delta @ y3.T
        dW3 = alpha * delta3 @ y2.T
        dW2 = alpha * delta2 @ y1.T
        dW1 = alpha * delta1 @ x.T
        W4 = W4 + dW4
        W3 = W3 + dW3
        W2 = W2 + dW2
        W1 = W1 + dW1
    return W1, W2, W3, W4
```

装 订 线

Listing 8: exp6_1.py：实验 6-1 验证样本与标签构造

```python
def make_dataset():
    X = np.zeros((5, 5, 5), dtype=float)

    X[:, :, 0] = [
        [0, 1, 1, 0, 0],
        [0, 0, 1, 0, 0],
        [0, 0, 1, 0, 0],
        [0, 0, 1, 0, 0],
        [0, 1, 1, 1, 0],
    ]
    X[:, :, 1] = [
        [1, 1, 1, 1, 0],
        [0, 0, 0, 0, 1],
        [0, 1, 1, 1, 0],
        [1, 0, 0, 0, 0],
        [1, 1, 1, 1, 1],
    ]
    X[:, :, 2] = [
        [1, 1, 1, 1, 0],
        [0, 0, 0, 0, 1],
        [0, 1, 1, 1, 0],
        [0, 0, 0, 0, 1],
        [1, 1, 1, 1, 0],
    ]
    X[:, :, 3] = [
        [0, 0, 0, 1, 0],
        [0, 0, 1, 1, 0],
        [0, 1, 0, 1, 0],
        [1, 1, 1, 1, 1],
        [0, 0, 0, 1, 0],
    ]
    X[:, :, 4] = [
        [1, 1, 1, 1, 1],
        [1, 0, 0, 0, 0],
        [1, 1, 1, 1, 0],
        [0, 0, 0, 0, 1],
        [1, 1, 1, 1, 0],
    ]

    D = np.eye(5, dtype=float)
    return X, D
```

装 订 线

Listing 9: exp6_1.py：实验 6-1 预测、评价与训练函数

```python
def predict(W1, W2, W3, W4, X):
    outputs = []
    for k in range(X.shape[2]):
        x = np.reshape(X[:, :, k], (25, 1))
        y1 = ReLU(W1 @ x)
        y2 = ReLU(W2 @ y1)
        y3 = ReLU(W3 @ y2)
        y = Softmax(W4 @ y3)
        outputs.append(y.reshape(-1))
    return np.array(outputs)
def predict_sigmoid(W1, W2, W3, W4, X):
    outputs = []
    for k in range(X.shape[2]):
        x = np.reshape(X[:, :, k], (25, 1))
        y1 = Sigmoid(W1 @ x)
        y2 = Sigmoid(W2 @ y1)
        y3 = Sigmoid(W3 @ y2)
        y = Softmax(W4 @ y3)
        outputs.append(y.reshape(-1))
    return np.array(outputs)
def mse(y, d):
    return float(np.mean((d - y) ** 2))
def accuracy(y, d):
    return float(np.mean(np.argmax(y, axis=1) == np.argmax(d, axis=1)))
def initialize_weights(seed=7):
    np.random.seed(seed)
    W1 = np.random.uniform(-1.0, 1.0, (20, 25))
    W2 = np.random.uniform(-1.0, 1.0, (20, 20))
    W3 = np.random.uniform(-1.0, 1.0, (20, 20))
    W4 = np.random.uniform(-1.0, 1.0, (5, 20))
    return W1, W2, W3, W4
def train(seed=7, epochs=3000, alpha=0.01):
    X, D = make_dataset()
    W1, W2, W3, W4 = initialize_weights(seed)
    history = []
    for _ in range(epochs):
        W1, W2, W3, W4 = DeepReLU(W1, W2, W3, W4, X, D, alpha=alpha)
        y = predict(W1, W2, W3, W4, X)
        history.append(mse(y, D))
    return X, D, W1, W2, W3, W4, np.array(history)
def train_sigmoid(seed=7, epochs=3000, alpha=0.01):
    X, D = make_dataset()
    W1, W2, W3, W4 = initialize_weights(seed)
    history = []
    for _ in range(epochs):
        W1, W2, W3, W4 = DeepSigmoid(W1, W2, W3, W4, X, D, alpha=alpha)
        y = predict_sigmoid(W1, W2, W3, W4, X)
        history.append(mse(y, D))
    return X, D, W1, W2, W3, W4, np.array(history)
```

Listing 10: exp6_1.py：实验 6-1 训练验证与结果输出

```python
def main():
    seed = 7
    epochs = 3000
    alpha = 0.01
    fig_dir = Path(__file__).resolve().parents[2] / "Lab8实验报告" / "figures"
    X, D, W1, W2, W3, W4, history = train(seed=seed, epochs=epochs, alpha=alpha)
    Y = predict(W1, W2, W3, W4, X)
    _, _, Ws1, Ws2, Ws3, Ws4, sigmoid_history = train_sigmoid(seed=seed, epochs=epochs, alpha=alpha)
    Y_sigmoid = predict_sigmoid(Ws1, Ws2, Ws3, Ws4, X)
    save_figures(fig_dir, X, D, Y, history, sigmoid_history=sigmoid_history)
    print("Experiment 6-1: Deep ReLU + Softmax network")
    print(f"seed={seed}, alpha={alpha}, epochs={epochs}")
    print(f"W1={W1.shape}, W2={W2.shape}, W3={W3.shape}, W4={W4.shape}")
    print(f"final MSE={history[-1]:.10f}, accuracy={accuracy(Y, D) * 100:.1f}%")
    print()
    for k, y in enumerate(Y):
        probs = " ".join(f"{value:.6f}" for value in y)
        print(f"sample {k + 1}: target={k + 1}, pred={np.argmax(y) + 1}, probs=[{probs}]")
    print()
    print("Sigmoid hidden-layer comparison")
    print(f"final MSE={sigmoid_history[-1]:.10f}, accuracy={accuracy(Y_sigmoid, D) * 100:.1f}%")
    print()
    print(f"figures saved to {fig_dir}")
if __name__ == "__main__":
    main()
```

4. 实验结果与分析

实验参数为：随机种子 7，学习率 $\alpha = 0.01$，训练 3000 epochs，初始权重从 $[-1, 1]$ 随机采样。相较于 XOR 实验中的浅层 Sigmoid 网络，本实验网络包含 4 层权重，梯度需要经过更长的链式反传，且 ReLU 输出不受 Sigmoid 上界压缩；因此采用更小的学习率 $\alpha = 0.01$ 保持更新稳定，3000 epochs 已足以使 ReLU 网络 MSE 收敛到 10^{-9} 量级。最终 MSE 为 3.9×10^{-9}，训练集分类准确率为 100%。为观察梯度消失的影响，额外保持网络结构、随机种子、学习率和训练轮数不变，仅将三个隐藏层激活函数替换为 Sigmoid 进行对比。

样本	目标类别	预测类别	类 1	类 2	类 3	类 4 / 类 5
1	1	1	0.999826	0.000010	0.000016	0.000051 / 0.000097
2	2	2	0.000011	0.999915	0.000000	0.000000 / 0.000074
3	3	3	0.000025	0.000012	0.999917	0.000001 / 0.000045
4	4	4	0.000065	0.000000	0.000001	0.999925 / 0.000010
5	5	5	0.000060	0.000034	0.000031	0.000000 / 0.999874

Table 4: 实验 6-1 最终 Softmax 输出

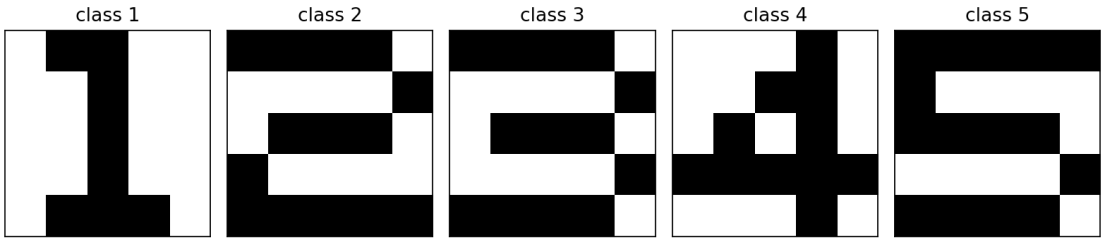


Figure 6: 实验 6-1：五个训练样本的 5x5 图案

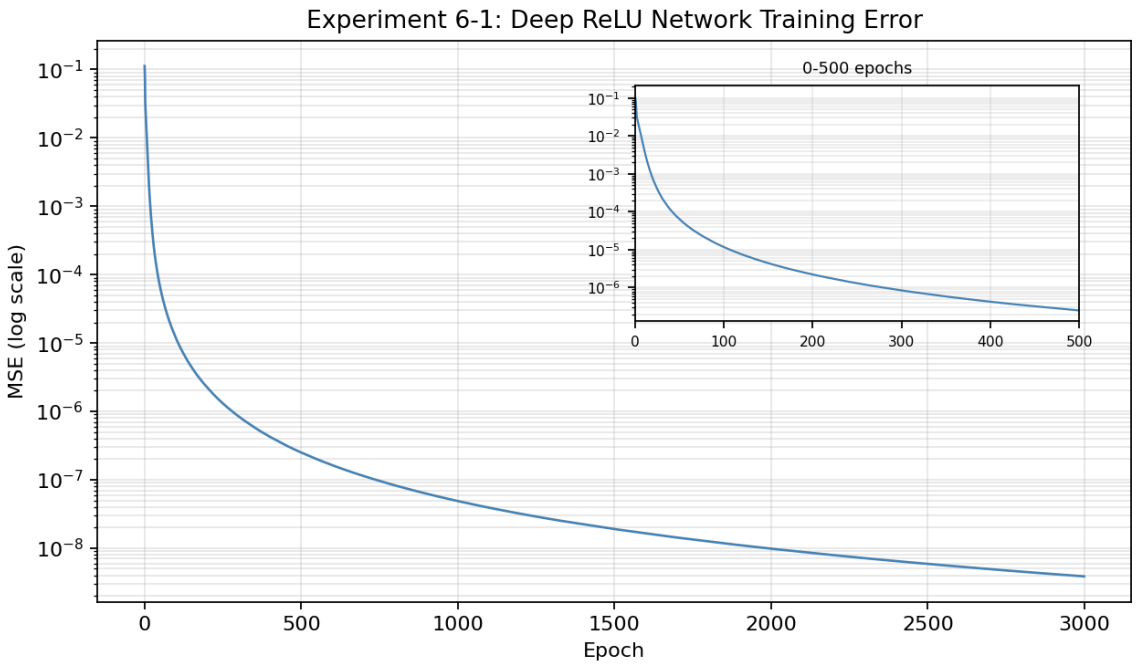


Figure 7: 实验 6-1：深度 ReLU 网络训练 MSE 曲线

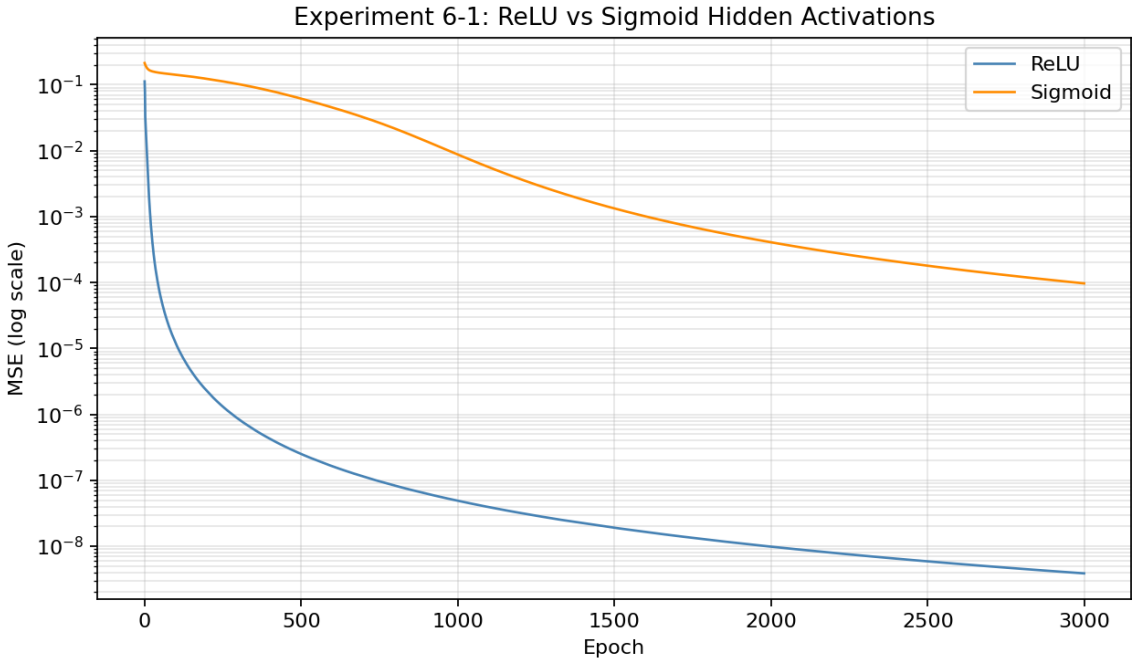


Figure 8: 实验 6-1：ReLU 与 Sigmoid 隐藏层 MSE 曲线对比

装 订 线

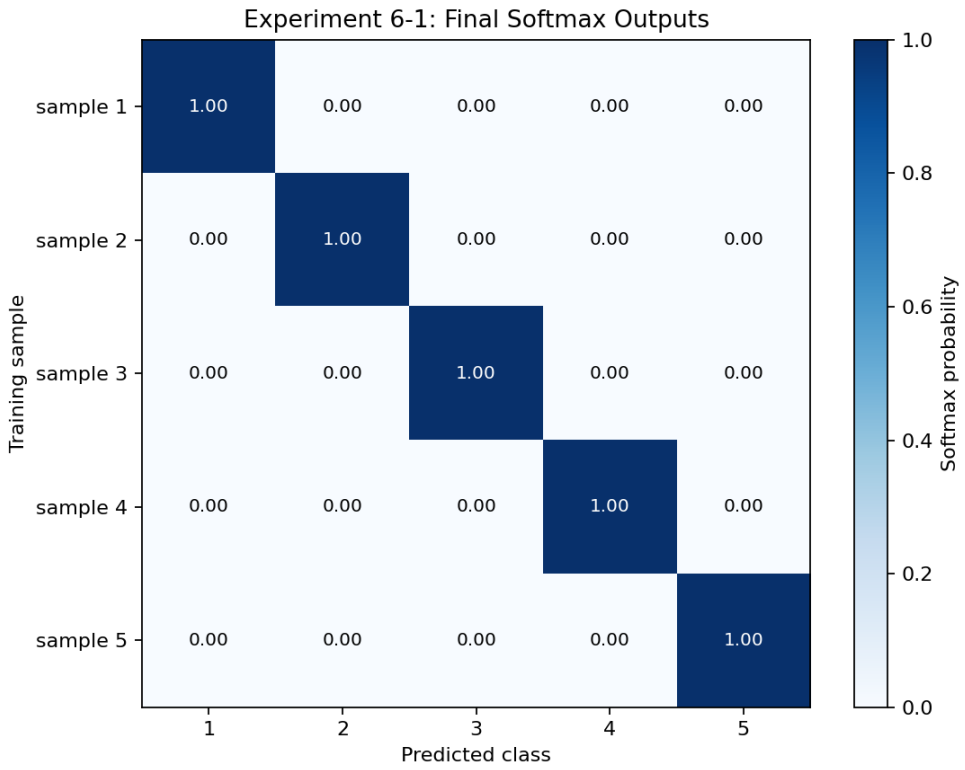


Figure 9: 实验 6-1：训练后 Softmax 输出概率矩阵

从表 4 和图 9 可以看出，训练后每个样本在正确类别上的 Softmax 概率均接近 1，在其他类别上的概率接近 0，说明深度 ReLU 网络已经成功记住并区分了这五个数字图案。图 7 显示 MSE 随 epoch 快速下降；其中 0–500 epoch 的放大子图展示了主要下降阶段，之后网络进入对数坐标下的缓慢精修。原因是当输出概率逐渐接近 one-hot 标签时，$e = d - y$ 整体变小，课件近似下的输出层 $\delta = e$ 也随之变小，权重更新幅度不断减弱；这与 Softmax+MSE 组合在接近饱和时梯度变小的现象一致。

图 8 进一步给出了 ReLU 与 Sigmoid 隐藏层的对比。两者最终训练集准确率均达到 100%，但 Sigmoid 网络最终 MSE 为 9.6964×10^{-5}，明显高于 ReLU 网络的 3.9×10^{-9}，约相差 2.5×10^{4} 倍。这说明在较深网络中，Sigmoid 导数最大仅为 0.25，多层链式相乘会明显削弱反传梯度，使权重更新更慢；ReLU 在正半轴导数为 1，因此更容易保持有效梯度并快速降低误差。

四、 综合总结

实验 5-2 说明了隐藏层对解决非线性可分问题的重要性。单层网络最终 MSE 约为 0.250071，输出基本贴近 0.5，按阈值得到的准确率并不代表模型真正学到了 XOR 规律；多层 BP 网络则将 MSE 降至 1.305×10^{-5}，并形成非线性决策边界。

实验 5-3 进一步验证了 momentum 对 BP 训练的影响。在相同随机种子与学习率下，$\beta = 0.9$ 将最终 MSE 从不用 momentum 时的 6.7×10^{-5} 降到 5.9×10^{-6}，约提升一个数量级；但 $\beta = 0.99$ 因等效步长过大而造成不稳定。将 α 降为 0.01 后又能恢复收敛，支持了等效学习率解释。

实验 6-1 将网络扩展到更深的 ReLU + Softmax 结构，完成了 5 类 5×5 图案识别任务，说明深度网络在层数增加后具备更强的特征表达和分类能力。将隐藏层 ReLU 替换为 Sigmoid 后，虽然最终仍能达到 100% 训练准确率，但 MSE 停留在 9.6964×10^{-5}，明显高于 ReLU 的 3.9×10^{-9}，从实验上体现了深层 Sigmoid 网络更容易因梯度衰减而收敛变慢。